Forget-IT: Optimal Good-Case Latency For Information-Theoretic BFT

Ittai Abraham
a16z Crypto Research
USA
ittaia@gmail.com

Sourav Das
Category Labs
USA
souravdas1547@gmail.com

Yuval Efron
Ritual
USA
efronyuv@gmail.com

Jovan Komatovic
Category Labs
USA
jovan.komatovic@gmail.com

Abstract

The good-case latency of a consensus protocol measures the latency from block proposal by a consensus leader to decision, in the case in which the leader is correct. It is arguably the efficiency metric most pertinent for discussing the practical latency performance of consensus protocols. Well understood in the context of the *authenticated* setting, with PBFT [Castro 99], Tendermint [Buchman 16] & Simplex [Chan, Pass 23] achieving the optimal good-case latency of 3 rounds, significant gaps remain in the *unauthenticated* setting. We present Forget-IT, an unauthenticated consensus protocol with optimal good-case latency of 3 rounds. Furthermore, our protocol only requires constant persistent storage, and has $O(n^2)$ message complexity per view.

CCS Concepts

• **Theory of computation → Distributed algorithms**.

Keywords

Information-theoretic Byzantine agreement, Partial synchrony, Good-case latency

ACM Reference Format:
Ittai Abraham, Sourav Das, Yuval Efron, and Jovan Komatovic. 2026. Forget-IT: Optimal Good-Case Latency For Information-Theoretic BFT. In *ACM Symposium on Principles of Distributed Computing (PODC '26), July 6–10, 2026, Egham, United Kingdom.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3796701.3815965

1 Introduction

Byzantine Fault Tolerant (BFT) consensus, the task of reaching agreement on a value among n processes, of which at most f are *corrupt*, is one of the most fundamental and well studied problems in the theory of distributed computing and cryptography [35]. The advent of blockchain technologies in the recent decade has highlighted not only its theoretical importance but also its practical impact. Consensus protocols deployed by modern blockchains almost always operate in the *partial synchrony* model, in which there is a liminal point in time, GST, such that the network is asynchronous prior to it and synchronous after it. Furthermore, they quite ubiquitously follow the *leader-based* paradigm: Time is divided into bounded intervals, referred to as *views*, with each having a designated *leader*, responsible with assembling and proposing the next block of transactions. In this work, we focus on the one-shot version of consensus, in which agreement on a single value must be reached by the processes. When discussing the efficiency of the leader-based approach in partial synchrony, two metrics dominate the conversation.

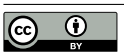
- **Good-case latency.** Roughly, good-case latency measures the worst-case time from the leader *entering* a view to *decision* by all correct processes, post GST, when the view leader is correct.
- **View length.** Roughly, view length measures the worst-case time from the last process entering a given view, to the last process leaving that view, post GST.

The motivation underlying these properties is that both aim to get a handle on the speed of convergence of the network post GST to a decision, and the power of the adversary in delaying that convergence. The study of these metrics dates back to the 1990s, with the seminal PBFT paper [23], presenting a protocol employing signatures (henceforth *authenticated*) which achieves good-case latency of 3δ (which is optimal [6]), where δ is the actual speed of the network after GST. In the context of the authenticated setting, the ideas of PBFT have been honed and refined over the years. Notable works include Tendermint and Simplex [15, 17]. Tendermint improves upon PBFT by reducing the communication complexity of each view to $O(n^2)$, down from $O(n^3)$. Furthermore, Tendermint boasts constant storage complexity of correct processes. Simplex also achieves $O(n^2)$ communication per view, and reduces the view length all the way down to 4Δ, where $\Delta \geq \delta$ is the known upper bound on message delays after GST.

On the other hand, our understanding of the *unauthenticated* setting, in which the adversary is full-information and computationally unbounded (henceforth *information-theoretical*), is lacking. In particular, the following question remained open:

> *Does there exist an information-theoretic BFT protocol with optimal resilience, optimal good-case latency, $O(n^2)$ communication per view, and $O(1)$ persistent storage?*

This question has been explicitly raised in several prior works, e.g., TetraBFT [45], IT-Hotstuff [9]. In fact, a discussion in TetraBFT



This work is licensed under a Creative Commons Attribution 4.0 International License.
PODC '26, Egham, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2512-8/26/07
https://doi.org/10.1145/3796701.3815965

entertains the idea of a barrier in the information-theoretic setting, citing the recent work of Attiya & Welch [12] as potential evidence that the good-case latency of 3, attainable with cryptographic tools, may be beyond reach in the information-theoretic setting (at least with constant storage).

Main result. We present Forget-IT, an information-theoretic Byzantine agreement protocol with optimal good-case latency of 3δ, secure against at most $f > 0$ Byzantine faults, where $3f + 1 = n$. Furthermore, our protocol, like TetraBFT, requires $O(1)$ words of persistent storage, $O(n^2)$ message complexity per view, and is optimistically responsive. This result settles the good-case latency of information-theoretic consensus with optimal resilience, and in addition shows that such good-case latency can be achieved with constant storage, low communication, and with no compromise on responsiveness.

Theorem 1 (See theorems 2 and 3). *There exists an information-theoretic, deterministic Byzantine agreement protocol that tolerates $f > 0$ faults with $n = 3f + 1$. The protocol achieves good-case latency of 3δ, view length of 7Δ, $O(1)$ persistent storage, and $O(n^2)$ message complexity per view, and is optimistically responsive.*

In fact, our main result obtains an even stronger notion of good-case latency, specifically that of *robust* good-case latency.

Robust good-case latency. Pondering about the notion of good-case latency in the information-theoretic setting for a moment, an astute reader might be inclined to reexamine the definition of good-case latency, and differentiate between two important variants.

(1) **Good-case latency**: The worst-case latency to decision, measured from $t = 0$, when the leader of the first view is correct and GST = 0.
(2) **Robust Good-case latency**: The worst-case latency to decision, measured from the first time a correct leader enters a view, post GST.

Clearly the latter metric can only be larger than the former, as robust good-case latency considers a strictly richer set of executions than good-case latency. Despite its stringency, the motivation for robust good-case latency is quite clear: Brief periods of asynchrony are common in practice, and robust good-case latency better captures the behavior of consensus protocols under such precarious conditions. Interestingly enough, the distinction between these two variants is rather obscured in the authenticated setting, as it is often the case that the two variants *coincide*: When the leader enters a view, it often has *cryptographic proof* that convinces the rest of the correct processes to catch up, and indeed these variants are equal for each of PBFT, Tendermint, and Simplex, with robust good-case latency of 3. Such proof, however, cannot exist in the information-theoretical (IT) setting, and indeed all existing protocols in the IT setting exhibit a significant gap between the two variants. In particular, to the best of our knowledge, in the IT setting, no peer-reviewed protocol achieves the optimal good-case latency of 3δ, and no protocol achieves the optimal robust good-case latency of 3δ, with the state of the art being TetraBFT, boasting good-case latency of 5δ, robust good-case latency of 7δ, and view length of 10Δ. Our protocol, Forget-IT, obtains the optimal robust good-case latency of 3δ and view length of 7Δ.

Related Work

Byzantine agreement. Byzantine agreement [1, 4, 16, 19, 20, 22, 31, 32, 34, 35, 38, 40, 42–44] is the problem of reaching agreement on a common value in a distributed system of n processes despite the presence of $f < n$ arbitrary failures. The seminal FLP [30] impossibility result showed that deterministic consensus cannot be solved in the asynchronous setting (where messages can be arbitrarily delayed), but can be solved in synchronous [25] and partially synchronous settings [23].

Latency of Byzantine agreement. Byzantine agreement protocols are primarily evaluated by *latency*, i.e., the number of rounds (or message delays) required for all correct processes to decide, which is the primary focus of this work. For Byzantine broadcast protocols, the deterministic Dolev-Strong protocol [25] solves broadcast in worst-case $f + 1$ rounds under synchrony, matching a lower bound [30]. For protocols with early stopping [24], a lower bound of $f' + 2$ rounds exists for $f' < f$ actual faults. The classic asynchronous Byzantine reliable broadcast by Bracha [14] has worst-case latency of 3 rounds, with subsequent work reducing the expected round complexity of Byzantine agreement to constant through randomization [2, 13, 18, 27, 29, 33, 36, 39, 41].

Good-case latency of Byzantine agreement. Since PBFT [23], which achieves good-case latency of 3δ, the study of good-case latency in Byzantine agreement has received significant attention. The work of Martin and Alvisi [37] was the first to achieve good-case latency of 2δ in partial synchrony, at the cost of requiring $n = 5f + 1$ processes. Tendermint [15] improved upon authenticated PBFT by reducing storage to $O(1)$ and communication to $O(n^2)$ per view. Following Abraham *et al.* [6], which established tight bounds on good-case latency for authenticated consensus in partial synchrony, research in this area has surged, with works exploring the problem across multiple dimensions [5, 7, 8, 11, 28].

In the unauthenticated setting, which is our primary focus, the two most closely related works are IT-HotStuff [9] and TetraBFT [45]. Both achieve constant persistent memory, $O(n^2)$ message complexity per view, and optimistic responsiveness.[1] However, neither achieves optimal good-case latency nor its robust variant, which is where our work advances the state of the art.

2 Technical Overview

We now provide a technical overview of the Forget-IT algorithm. Forget-IT has the standard *leader-based* structure and proceeds in *views*. Each view V has a designated leader and has three implicit algorithmic components: *view core*, *view change*, and *view transition*. The protocol's logic, explained in this section, is illustrated in Figure 1.

View core. The view core component is the decision-making logic within a single view. This has a structure similar to the normal case operation in the classic PBFT protocol [23]. As in PBFT, the view core's part of each view V has three rounds. The leader of the view starts by proposing a value v, i.e., it broadcasts v to all. Upon receiving the proposal v from the leader, the processes engage in two rounds of voting to potentially decide on the value.

[1] Throughout this work, we use the terms "storage" and "memory" interchangeably.

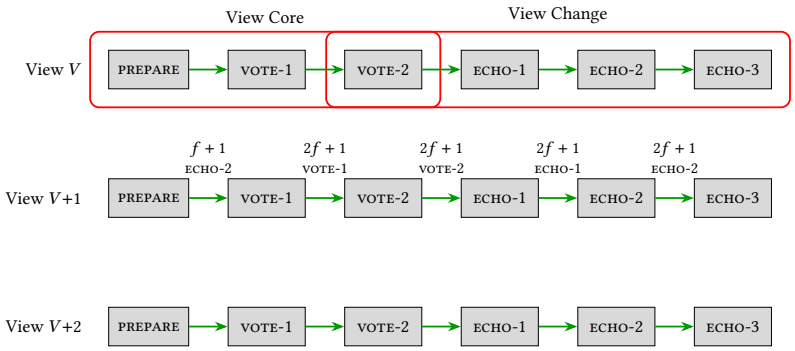


Figure 1: Flowchart of Forget-IT's progress in any given view. A view commenced with a leader proposing a value v via a PREPARE message. If deemed safe, e.g., by seeing $f+1$ ECHO-2 messages for v from the previous view, correct processes multicast a VOTE-1 message for v. A quorum of those for v triggers a VOTE-2 message for v. A quorum of those for v triggers a decision. If a decision was not triggered, then a quorum of VOTE-2 messages triggers the view-change protocol, where processes exchange ECHO-1, ECHO-2, and ECHO-3 messages. A process proceeds to the next view upon collecting (1) a quorum of ECHO-3 messages, or (2) $f+1$ ECHO-2 messages coupled with a PREPARE message from the next leader.

Table 1: A table of previous information-theoretic protocols and their corresponding metrics, juxtapositioned against our protocol Forget-IT.

	Good-case latency	Robust good-case latency	View length	Persistent storage / Message complexity	Responsiveness
PBFT [23]†	3δ	7δ	9Δ	$O(1)/O(n^4)$	✓
IT-HS (blog) [10]‡	4δ	6δ	7Δ	$O(1)/O(n^2)$	×
IT-HS [9]	6δ	8δ	10Δ	$O(1)/O(n^2)$	✓
TetraBFT [45]	5δ	7δ	10Δ	$O(1)/O(n^2)$	✓
Forget-IT	3δ	3δ	7Δ	$O(1)/O(n^2)$	✓

† Suggested protocol, not peer-reviewed or proven.
‡ Although the leader must wait 2Δ before proposing (sacrificing responsiveness), the (robust) good-case latency measures time from the issued proposal, hence the 2Δ wait does not appear in the good-case latency metrics.

In signature-based protocols, processes vote on a proposal only if the proposal is *justified*, i.e., processes can locally determine that voting on the proposed value would not cause a safety violation [23]. Typically, this justification consists of a quorum certificate: information about the internal states of $2f+1$ processes, authenticated with their digital signatures. Since we are in the information-theoretic (IT) setting, we cannot rely on signature-based justification. Thus, we introduce the notion of *guards*, a local state each process maintains to succinctly represent information regarding potential decisions in previous views. The guard ensures that if any correct process decides a value v in some view V, then all correct processes will update their guards to the value v. In more detail, our protocol relies on the guarding mechanism as follows.

Let $guard_i[V]$ be the guard at the i-th process p_i in a given view V. Upon receiving the proposal from the leader consisting of the value v, process p_i casts its vote for v, i.e., sends VOTE-1 message to all, only if either $guard_i[V] = \mathsf{SAFE}$ or $guard_i[V] = v$. Here, SAFE is a default wildcard constant of the protocol, and $guard_i[V] = \mathsf{SAFE}$ indicates that no processes could have decided any value in prior views, and thus voting for the current proposal would not violate safety. Similarly, $guard_i[V] = v$ indicates that some other process might have decided v during a previous view. Thus, voting on value v (and only on value v) in this current view is safe.

Upon receiving VOTE-1 messages from $2f+1$ processes for a proposal with value v, as in PBFT, each process *locks* on v, and casts their second vote by broadcasting their VOTE-2 message for the value v. (Looking ahead, to improve the latency of the view-change procedure, each process p_i in view V also includes $guard_i[V]$ in its VOTE-2 message, but we can omit these details for now.) Finally, upon receiving $2f+1$ VOTE-2 message for the same value v, each process *decides* v.

View change. As in PBFT, when a view does not succeed (due to a faulty leader or network delays), our protocol enters a logical view-change phase. This is the most technically involved and novel component of Forget-IT, as it must ensure that correct processes make progress while preserving previous decisions without relying on cryptography. Let us elaborate.

Correct processes initiate the view-change phase when their local view does not make progress for 3Δ time. A correct process initiates the view change phase by broadcasting a VOTE-2 message, unless it has already sent a VOTE-2 message for that view as part of the normal case operation (in the view core). We note that the VOTE-2 message in Forget-IT serves two purposes: first, to drive decisions in a view, and second, if the situation arises, to assist the view-change phase, as follows.

Acceptable VOTE-2 messages for tentative assessment of agreement. Each process waits to receive $2f+1$ *acceptable* VOTE-2 message, where the acceptable notion seeks to prevent malicious processes

from sending VOTE-2 messages for arbitrary values. However, unlike a signature-based protocol, we cannot rely on signatures to let processes make their VOTE-2 messages acceptable. Therefore, we introduce a message-based acceptance condition. In Forget-IT, a process deems a VOTE-2 message for a value v acceptable only upon receiving $f + 1$ VOTE-1 messages for the same value v. This rule ensures that correct processes accept a VOTE-2 message for v only if at least one correct process previously sent a VOTE-1 message for v.

We then combine this acceptance condition with the following properties: (i) once any correct process decides a value v, all correct processes update their guards to v at the beginning of the next view; and (ii) a correct process sends a VOTE-1 message only when the proposed value matches its current guard. Together, these properties imply that if any correct process decides v, then no correct process will ever accept a VOTE-2 message for any value $v' \neq v$ in any future view. In the remainder of this section, we assume that correct processes handle only acceptable VOTE-2 messages.

Thus, once a process receives a quorum of $2f+1$ VOTE-2 messages and aims to start the view-change procedure, it first analyzes them to determine locally whether some value may have been decided in the current view. It then disseminates this assessment to all other processes using a three-round echo protocol. If the process concludes that no decision could have occurred in the current view, it propagates this negative assessment using the same protocol. Crucially, the processes make only tentative assessments of the decision in the current view and can revise these assessments upon receiving additional VOTE-2 messages. For example, a process may initially receive $2f + 1$ VOTE-2 messages and infer that a value v might have been decided, and accordingly broadcast v. If it later receives additional VOTE-2 messages that invalidate this inference, it updates its assessment to "no decision occurred" and broadcasts the revised assessment.

Looking ahead, this revision mechanism is essential for liveness. Without it, a Byzantine adversary could selectively release VOTE-2 messages to cause different correct processes to form conflicting assessments and indefinitely stall the protocol's progress.

Three-phase echo protocol. Upon determining whether a decision has occurred in the current view, each process invokes a *three-phase echo protocol* to verifiably propagate this assessment across views, while maintaining constant persistent memory and enabling low-latency view transitions.

In the first echo phase, a process broadcasts an ECHO-1 message containing the value v that it tentatively considers decided in the current view. A process may later revise this assessment and, upon doing so, broadcasts a new ECHO-1 message reflecting the updated opinion. A naive implementation of this approach would allow up to $O(n)$ such revisions per process, leading to a total message complexity of $O(n^3)$. To avoid this blowup, our protocol restricts each process to sending at most two ECHO-1 messages per view. Furthermore, a process may use its second ECHO-1 message only to broadcast the wildcard value SAFE, which indicates that all values are safe.

In the second echo phase, each process waits to receive a quorum of ECHO-1 messages for some value v. Upon collecting such a quorum, it confirms v by broadcasting an ECHO-2 message for v. Importantly, in Forget-IT, we count ECHO-1 messages carrying the wildcard value SAFE towards a quorum of any value $v \neq \perp$ (as they indicate that all values, including v, are safe). Moreover, as in the first echo phase, each process sends at most two ECHO-2 messages per view, where it may use the second only for the wildcard value SAFE.

In the third echo phase, each process waits to receive a quorum of ECHO-2 messages for some value v. Upon collecting such a quorum, it confirms v by broadcasting an ECHO-3 message for v. As before, we count ECHO-2 messages carrying the wildcard value SAFE toward the quorum for any value. Moreover, each process sends at most two ECHO-3 messages per view, and may use the second only for the wildcard value SAFE.

View transitioning. The view-transition component specifies three mechanisms by which processes in Forget-IT enter a new view.

First, processes enter view $V + 1$ using a quorum of ECHO-3 messages from view V. A non-leader process enters the next view after receiving $2f + 1$ ECHO-3 messages. In contrast, the leader of the next view enters upon receiving only $f + 1$ ECHO-3 messages. We introduce this asymmetric quorum requirement to allow the next leader to enter its view "quickly" after the view starts. Specifically, once a non-leader enters a view after collecting $2f + 1$ ECHO-3 messages, the next leader receives at least $f + 1$ of those messages within δ time and thus also enters the view within that time. In comparison, the standard ready amplification technique of Bracha [14] guarantees that the next leader enters the view only within 2δ time from the first correct process entering the new view.

Second, a process enters a new view V upon receiving a proposal from the leader of the V and if the process receives $f + 1$ ECHO-2 messages from view $V - 1$. This mechanism ensures that every correct process enters the leader's view within δ time of the leader entering view V. To see this, note that the leader enters the view V upon receiving $f + 1$ ECHO-3 message. This implies that at least one correct process has sent ECHO-3. Since a correct process sends an ECHO-3 message only upon receiving $2f + 1$ ECHO-2 messages, this implies that all correct processes will receive at least $f + 1$ ECHO-2 messages within δ time from the leader entering the view. Crucially, the received ECHO-2 messages also serve as justification for the leader's proposal, enabling liveness (and optimistic responsiveness).

Third, a process in view V' may *passively* enter a future view $V > V'$ upon receiving $f + 1$ ECHO-1 messages for view V. This is termed passive entry because the process skips the view core and proceeds directly to view-change procedure of view V (to transition to view $V+1$). We introduce this passive entry mechanism specifically to achieve $O(1)$ persistent storage, as detailed in §4.

Updating guards at processes. The last critical step of Forget-IT is the procedure for processes to update their guards. Guard updates are critical as guards prevent correct processes from voting on conflicting proposals (safety), while enabling processes to accept new proposals consistent with prior decisions (liveness). In a nutshell, processes update their guards during view transitions based on the quorum of echo messages that triggered the transition.

Challenges due to $O(1)$ persistent storage requirements. Requiring each process to maintain only $O(1)$ persistent storage introduces fundamental challenges. In particular, we cannot assume reliable delivery of an unbounded number of messages between a

pair of correct processes. Reliable delivery requires the sender to retain all in-transit messages in persistent memory until it receives confirmation of delivery. Under an $O(1)$ storage bound, such buffering is impossible. This limitation motivates the need and justifies our new definition of a formal model (see §3) that explicitly captures protocols operating under constant persistent storage constraints.

A second challenge is to ensure that the $O(1)$ state maintained by each process effectively compresses all relevant information from prior views. In Forget-IT, each process keeps track of only a small, constant number of recent views. Designing a protocol with this property is non-trivial. For example, once a correct process enters a view V, the protocol must ensure that no correct but slow process is left behind in a much earlier view. Otherwise, the system would either sacrifice liveness or require processes to retain state for arbitrarily old views, violating the storage bound. Our multi-level echo structure directly addresses this issue. By carefully propagating and amplifying view-transition information associated with only a few recent views (rather than all views), the protocol guarantees that slow processes can catch up while allowing correct processes to safely discard information regarding old views.

Summary of our results. Combining all the above, we get a partially synchronous consensus protocol in the IT setting with $O(n^2)$ message complexity per view, and $O(1)$ persistent storage. Moreover, the protocol is responsive and has a good-case latency of 3δ, i.e., when the leader is correct, processes decide on a value using three rounds of communication. The protocol also has a robust good-case latency of 3δ, which matches the robust case latency of state-of-the-art consensus protocols that rely on digital signatures, and, finally, a view duration of 7Δ.

3 Preliminaries

Processes & the adversary. We consider a static message-passing system $\{p_1, p_2, \ldots, p_n\}$ consisting of $n = 3f + 1$ processes, for some integer $f > 0$. Each process is modeled as a deterministic state machine equipped with a local clock. An *adversary* may corrupt up to f processes at arbitrary times during the execution; we refer to such an adversary as f-bounded.[2] Corrupted processes are Byzantine and may behave arbitrarily, whereas non-corrupted processes follow their state machines, as prescribed by the protocol; we refer to them as *faulty* and *correct*, respectively. Moreover, we assume that the adversary is (1) *full-information*, meaning that it observes the internal state of every process, and (2) *computationally unbounded*. Finally, we model local computation as taking zero time, since it is negligible compared to message delays.

Communication network. We assume a point-to-point message-passing network with *authenticated channels*, meaning that the recipient of each message can verify the sender's identity. However, we do not assume that channels are *reliable*, i.e., we do not assume that every sent message is eventually received. This modeling choice is deliberate, and plays a crucial part in how we formally handle memory complexity. Indeed, reliable channels require the sender to buffer every message until delivery is confirmed, which necessitates unbounded memory and is therefore incompatible with the constant-memory goal of our protocol.

Instead, we adopt an unreliable network model in which communication proceeds exclusively via broadcasts: each broadcast operation unicasts the message separately to every process, retransmitting it indefinitely until the broadcasting operation is explicitly *abandoned*, following the semantics of [26]. Under this model, we say that a message m is *alive* at time τ if the broadcast of m has been initiated by time τ and has not been abandoned by time τ. We say that m is *forever-alive* if its broadcast is initiated at some point and never abandoned. A forever-alive message is *guaranteed* to be delivered by every correct process, as its indefinite retransmission ensures delivery despite the lack of reliable channels.

Persistent memory. To reason formally about the memory usage of our protocol, we incorporate persistent memory directly into the model. Each process's state machine includes a *persistent memory* component, together with rules governing how this memory is updated over time. Intuitively, persistent memory is the only state that survives crashes.

Formally, in addition to Byzantine corruptions, the adversary may temporarily *crash* a correct process, with the guarantee that every crashed correct process eventually *recovers*. While crashed, a process performs no local computation and sends no messages; in particular, all broadcasting operations initiated before the crash are implicitly abandoned. Crucially, however, the process's persistent memory remains unchanged during the crash. Upon recovery, the process executes a designated *recovery procedure*—a purely local computation that takes as input only the contents of persistent memory and has access to no other information, neither volatile state nor previously received messages. The recovery procedure produces a new local state, after which the process resumes normal execution according to its state machine. To meaningfully reason about sub-linear (in n) storage, we assume that the identities of participating processes are stored in a read-only part of the state machine which remains resilient to crashes, and is not counted towards the size of a process's persistent memory.

Partial synchrony. We consider the standard partially synchronous model. Specifically, there exists an unknown Global Stabilization Time (GST), chosen by the adversary, that separates an initial asynchronous period from a subsequent synchronous one. The synchronous period is governed by a known[3] upper bound Δ on message delays: any message broadcast at time τ and alive throughout $[\tau, \max(\tau, \text{GST}) + \Delta]$ is received by time $\max(\tau, \text{GST}) + \Delta$. We denote by $\delta \leq \Delta$ the actual message delay after GST and assume that any message broadcast at time τ and alive throughout $[\tau, \max(\tau, \text{GST}) + \delta]$ is received by time $\max(\tau, \text{GST}) + \delta$ (See *Responsiveness* below for a formal definition).

We further assume that all correct processes begin executing the protocol by GST, and that hardware clocks, while subject to arbitrary drift before GST, do not drift after GST. Finally, we assume that the adversary may crash correct processes only before GST, and that every crashed correct process also recovers before GST. Consequently, from GST onward, all correct processes remain continuously online.

Byzantine agreement & its metrics. In this work, we design a Byzantine agreement protocol that operates in the model described

[2] Note that for deterministic protocols, static and adaptive adversaries are equivalent.

[3] I.e., all correct parties know the value Δ.

above. Formally, we solve the problem in which every correct process proposes an initial value and aims to decide a value while satisfying the following properties:

- *Agreement:* No two correct processes decide different values.
- *Termination:* Every correct process eventually decides.
- *Non-triviality:* If all processes are correct, all propose the same value v, and GST = 0, then no correct process decides any value different from v.

We note that while various other validity constraints exist [21], we deliberately choose the simplest formulation to focus attention on the core algorithmic challenges, which are independent of the specific validity variant. However, we emphasize that our protocol can be straightforwardly adapted to satisfy external validity [15, 44], arguably the most important validity property, which requires that any decided value must be valid according to some predetermined logical predicate. The only modification needed is for processes to support only valid values. We further note that our choice to focus on the one-shot version of the consensus task (I.e., BA) allows us to more coherently illustrate our main techniques, the protocol can be straightforwardly extended to provide a solution to the Atomic Broadcast problem, while maintaining its communication good-case latency, and responsiveness properties.

Efficiency metrics. We evaluate the efficiency of protocols using three key metrics. For an f-bounded adversary $\mathcal{A}$ (capable of corrupting up to f processes) and an f-secure (resilient to f corrupted processes) leader-based Byzantine agreement protocol Π, we denote by $\mathsf{DL}_{\mathcal{A}}(\Pi)$ the smallest time T by which every correct process decides (but not necessarily terminates) in the presence of $\mathcal{A}$, normalized to GST, i.e., $\mathsf{DL}_{\mathcal{A}}(\Pi) = T - \mathrm{GST}(\mathcal{A})$. (Here, given an adversary $\mathcal{A}$, $\mathrm{GST}(\mathcal{A})$ denotes its GST.) For a family $\mathcal{F}$ of f-bounded adversaries, we define the decision latency of Π with respect to $\mathcal{F}$ to be $\mathsf{DL}_{\mathcal{F}}(\Pi) = \sup_{\mathcal{A}\in\mathcal{F}} \mathsf{DL}_{\mathcal{A}}(\Pi)$.

- *Good-case latency* T_{gc}*:* Consider the family $\mathcal{F}_{\mathrm{gc}}$ of f-bounded adversaries $\mathcal{A}$ for which the first leader ℓ is correct and GST = 0. We define $\mathsf{T}_{\mathrm{gc}} \triangleq \mathsf{DL}_{\mathcal{F}_{\mathrm{gc}}}(\Pi)$.
- *Robust good-case latency* $\mathsf{T}_{\mathrm{rgc}}$*:* Consider the family $\mathcal{F}_{\mathrm{rgc}}$ of f-bounded adversaries $\mathcal{A}$ for which GST coincides with a correct leader entering its view. We define $\mathsf{T}_{\mathrm{rgc}} \triangleq \mathsf{DL}_{\mathcal{F}_{\mathrm{rgc}}}(\Pi)$.
- *Memory complexity* $\mathsf{PM}(\Pi)$*:* Let $\mathsf{PM}_{\mathcal{A}}(p_i, \tau)$ be the size (in words) of the persistent memory register of a correct process p_i at time τ under adversary $\mathcal{A}$, where each word can store a constant number of messages, values, and counters. The memory complexity of Π with respect to $\mathcal{A}$ is $\mathsf{PM}_{\mathcal{A}}(\Pi) = \max_{\text{correct } p_i, \tau\in\mathbb{N}} \mathsf{PM}_{\mathcal{A}}(p_i, \tau)$, and the overall memory complexity of Π is $\mathsf{PM}(\Pi) = \max_{f\text{-bounded } \mathcal{A}} \mathsf{PM}_{\mathcal{A}}(\Pi)$.

We refer the reader to the full version [3] for a formal definition of the view length metric, as it is less central to our results than the above metrics.

Responsiveness. With the definition of good-case latency in mind, we can now define the notion of *responsiveness*. For a BA protocol Π, Let $E(\Pi)$ denote an execution of the protocol, which is entirely determined by the adversary's $\mathcal{A}$ choices of corrupt parties & their behaviour, GST, and network delays within the realm of the partially synchronous model. For an execution $E(\Pi)$, and a message m from correct party p to correct party q, we denote by $\delta(m)$ the wall-clock time difference between p sending m and q receiving m. Denoting by M_E the set of messages sent between correct parties in E, we define $\delta(E) = \max_{m\in M_E} \delta(m)$ to be the *actual delay* of the execution E. We say that a protocol Π is *responsive* if its good-case latency is dependent only on δ. More formally, if there exists a constant $c > 0$ such that for every execution $E \in \mathcal{F}_{\mathrm{gc}}$, $\mathsf{DL}_E(\Pi) < c \cdot \delta(E)$.

4 Forget-IT: Pseudocode & Proof Sketch

We begin the presentation of Forget-IT with a description of the pseudocode (§4.1), followed by an informal analysis of correctness and complexity (§4.2). We refer the reader to the full version [3] for the formal proof.

4.1 Pseudocode

The pseudocode is given in Algorithm 1. In essence, execution unfolds in a sequence of *views* $V = 1, 2, 3, \ldots$, each with a designated *leader* that rotates in round-robin fashion. Within each view, the leader proposes a value and the remaining processes attempt to reach agreement on it. If this attempt fails (due to a faulty leader or network instability), the protocol transitions to the next view, carrying forward enough information to preserve both safety and liveness.

As discussed in §2, Forget-IT comprises three interacting algorithmic components:

- *View core:* The decision-making logic within a single view.
- *View change:* The mechanism for determining, once a view completes, a safe and live value to carry into the next view.
- *View transitioning:* The rules governing when processes leave their current view and enter a new one.

To build intuition, we describe each component in detail through the lens of a fixed view V: how processes attempt to decide in V (view core), how they distill the outcome of V into information for $V + 1$ (view change), and how they move from V to $V + 1$ (view transitioning).

4.1.1 View Core. The view core of view V follows a standard three-round "propose–vote–decide" structure. Upon entering view V, the leader broadcasts a $\langle \textsc{prepare}, V, v\rangle$ message carrying its proposed value v (line 34); how the leader determines value v is discussed in §4.1.3. Each process p_i that receives the proposal checks it against $guard_i[V]$, its local guard for view V (line 36), a compact summary of what p_i knows about potential decisions in prior views. A guard can take two forms: SAFE, a special constant indicating that no value could have been decided previously (thus any proposal is safe to support), or some specific value v, indicating that v is the only value that could have been decided (thus restricting p_i to supporting only v). Guards are updated during the view transition phase; we describe this mechanism in §4.1.3. Importantly, within a view, a process may update its guard from a specific value to SAFE upon learning that all values are safe. Hence, a correct process guarding a specific value might initially ignore the leader's proposal for some other value as it does not deem it safe; however, later in the same view, through the rule at line 50, it can learn that the leader's proposal (and, in general, all values) are safe, update its guard to SAFE (line 51), and then support the proposal.

Once the guard check passes, each process broadcasts the message $\langle \textsc{vote-1}, V, v\rangle$ (line 38). Upon collecting $2f+1$ matching VOTE-1

Algorithm 1 Forget-IT: Pseudocode (for process p_i) [part 1 of 3]

Constants:
String SAFE = 'Every value is safe' ▷ a special string indicating that all values are safe
Global variables (persistent memory):
Value $proposal_i$ ▷ p_i's proposal
View $view_i$ ▷ p_i's current view
View-specific variables (persistent memory), indexed by view V:
Value ∪ {SAFE} $guard_i[V] \leftarrow \perp$ ▷ p_i's guarded value for view V
Value $value_to_propose_i[V] \leftarrow \perp$ ▷ value to propose if p_i is the leader of view V
Value $lock_i[V] \leftarrow \perp$ ▷ p_i's locked value for view V (or $\perp$ if none)
Message $vote1_i[V]$, $vote2_i[V] \leftarrow \perp$ ▷ VOTE-1 and VOTE-2 messages broadcast by p_i in view V
Value ∪ {SAFE} $potential_decision_i[V] \leftarrow \perp$ ▷ used for view change
Boolean $decision_alert_i[V] \leftarrow \perp$ ▷ tracks whether a decision might have occurred in view V
Boolean $vc_i[V] \leftarrow false$ ▷ whether the view-change procedure for view V has started
Echo variables (persistent memory), indexed by view V and $k \in \{1,2\}$:
Message $echo1_i[V][k]$, $echo2_i[V][k]$, $echo3_i[V][k] \leftarrow \perp$ ▷ ECHO-1, ECHO-2, ECHO-3 messages broadcast by p_i in view V
Value ∪ {SAFE} $val_echo1_i[V][k]$, $val_echo2_i[V][k]$, $val_echo3_i[V][k] \leftarrow \perp$ ▷ values from the ECHO messages
Rules:
- Process p_i maintains (in persistent memory) view-specific and echo variables for views $view_i$, $view_i - 1$, and $view_i - 2$ only.
- Upon entering view V, abandon all ongoing broadcasts except for messages from views $V-2$ and $V-1$.
Notation:
- "Receipt of k messages" means receipt from k *different* processes.
Rules:
- A received message $\langle$VOTE-2, $V, v, \cdot\rangle$ with $v \neq \perp$ is *accepted* only after p_i has received $f+1$ messages $\langle$VOTE-1, $V, v\rangle$.
- A received message $\langle$VOTE-2, $V, \perp, \cdot\rangle$ is *accepted* immediately upon reception.
- For each view V, at most one $\langle$VOTE-2, $V, \cdot, \cdot\rangle$ message is accepted from each process.
upon starting:
$proposal_i \leftarrow p_i$'s proposal; $value_to_propose_i[1] \leftarrow proposal_i$
$guard_i[1] \leftarrow$ SAFE ▷ initialize p_i's guard to SAFE
enter view 1 ▷ active entry
upon entering view V:
$view_i \leftarrow V$ ▷ update the current view
invoke ensure_view_change$(V-2, guard_i[V])$ ▷ ensure view-change for view $V-2$ is active
invoke ensure_view_change$(V-1, guard_i[V])$ ▷ ensure view-change for view $V-1$ is active
if $p_i =$ leader(V): **broadcast** $\langle$PREPARE, V, $value_to_propose_i[V]\rangle$ ▷ leader proposes a value
upon receipt of $\langle$PREPARE, $view_i, v\rangle$ **from** leader$(view_i)$:
wait until $guard_i[view_i] =$ SAFE **or** $guard_i[view_i] = v$ ▷ check if it is safe to support the leader's proposal
if $vote1_i[view_i] = \perp$ **and** $vc_i[view_i] = false$:
$vote1_i[view_i] \leftarrow \langle$VOTE-1, $view_i, v\rangle$; **broadcast** $vote1_i[view_i]$
upon receipt of $2f+1$ $\langle$VOTE-1, $view_i, v\rangle$ messages for the same value v **and** $vc_i[view_i] = false$:
if $vote2_i[view_i] = \perp$:
$lock_i[view_i] \leftarrow v$ ▷ lock on value v
$vote2_i[view_i] \leftarrow \langle$VOTE-2, $view_i, lock_i[view_i], guard_i[view_i]\rangle$; **broadcast** $vote2_i[view_i]$
upon receipt of $2f+1$ $\langle$VOTE-2, $view_i, v, \cdot\rangle$ messages for the same value $v \neq \perp$:
if p_i has not decided yet: **trigger** decide(v) ▷ decide on v
upon $view_i$ is (non-passively) entered **and** $1\Delta + 2\Delta$ time has elapsed since (non-passively) entering view $view_i$:
if $vote2_i[view_i] = \perp$:
▷ timeout: broadcast VOTE-2 even without quorum to enable view change
$vote2_i[view_i] \leftarrow \langle$VOTE-2, $view_i, lock_i[view_i], guard_i[view_i]\rangle$; **broadcast** $vote2_i[view_i]$
▷ once this condition holds, p_i learns that any value is safe and clears its guard
upon cert(ECHO-2, $f+1, view_i - 1, v) = true$ for some $v \in$ Value ∪ {SAFE} **and** $guard_i[view_i] \notin \{$SAFE, $v\}$:
$guard_i[view_i] \leftarrow$ SAFE

Algorithm 2 Forget-IT: Pseudocode (for process p_i) [part 2 of 3]

upon recovery: ▷ recovery procedure
$vc_i[view_i] \leftarrow true$; $vc_i[view_i - 1] \leftarrow true$; $vc_i[view_i - 2] \leftarrow true$
if $vote2_i[view_i] = \perp$: $vote2_i[view_i] \leftarrow \langle$VOTE-2, $view_i, lock_i[view_i], guard_i[view_i]\rangle$
for each view $V \in \{view_i - 2, view_i - 1, view_i\}$ and message $m \in \{vote1_i[V], vote2_i[V]\}$ **with** $m \neq \perp$: **broadcast** m
for each $V \in \{view_i - 2, view_i - 1, view_i\}$ and $k \in \{1,2\}$:
for each message $m \in \{echo1_i[V][k], echo2_i[V][k], echo3_i[V][k]\}$ **with** $m \neq \perp$: **broadcast** m
upon existence of a set S of $2f+1$ accepted $\langle$VOTE-2, $V, \cdot, \cdot\rangle$ messages such that all non-$\perp$ lock values (if any) in S are identical **and** $V \in \{view_i - 2, view_i - 1, view_i\}$ **and** $decision_alert_i[V] = \perp$:
if there exists a non-$\perp$ lock value ℓ in S: $decision_alert_i[V] \leftarrow true$; $potential_decision_i[V] \leftarrow \ell$
else $decision_alert_i[V] \leftarrow false$ ▷ no locks exist, so all values are safe
upon acceptance of $2f+1$ $\langle$VOTE-2, $V, \perp, \cdot\rangle$ messages **and** $V \in \{view_i - 2, view_i - 1, view_i\}$ **and** $decision_alert_i[V] = true$:
$decision_alert_i[V] \leftarrow false$ ▷ if a quorum has no locks, no decision occurred in view V
upon existence of a set S of $2f+1$ accepted $\langle$VOTE-2, $V, \cdot, \cdot\rangle$ messages **such that** all non-$\perp$ lock values in S differ from $potential_decision_i[V]$ **and** $V \in \{view_i - 2, view_i - 1, view_i\}$ **and** $decision_alert_i[V] = true$:
$decision_alert_i[V] \leftarrow false$ ▷ no decision occurred in view V
upon $1\Delta + 3\Delta$ time has elapsed since entering view $view_i$:
$vc_i[view_i] \leftarrow true$ ▷ the view-change procedure for $view_i$ starts
upon acceptance of $\langle$VOTE-2, $V, v \neq \perp, \cdot\rangle$ **and** $V \in \{view_i - 2, view_i - 1, view_i\}$ **and** ($v = potential_decision_i[V]$ **or** $decision_alert_i[V] = false$) **and** $vc_i[V] = true$:
invoke send_echo_one(V, v) ▷ v is safe based on a received lock, issue ECHO-1 for it
upon acceptance of $f+1$ $\langle$VOTE-2, $V, \cdot, G\rangle$ messages for the same $G \in$ Value∪{SAFE} **and** $V \in \{view_i - 2, view_i - 1, view_i\}$ **and** $decision_alert_i[V] = false$ **and** $vc_i[V] = true$:
invoke send_echo_one(V, G) ▷ v is safe based on received guards, issue ECHO-1 for it
upon $decision_alert_i[V] = false$ **and** $V \in \{view_i - 2, view_i - 1, view_i\}$ **and** divergent_guards$(V) = true$ **and** $vc_i[V] = true$:
invoke send_echo_one$(V,$ SAFE$)$ ▷ correct processes hold different guards (implying all values are safe), issue ECHO-1 for SAFE
upon cert(ECHO-1, $2f+1, V, v) = true$ for some $v \in$ Value ∪ {SAFE} and some $V \in \{view_i - 2, view_i - 1, view_i\}$:
invoke send_echo_two(V, v) ▷ a quorum of ECHO-1 message for v is received, issue ECHO-2 for v
upon cert(ECHO-2, $2f+1, V, v) = true$ for some $v \in$ Value ∪ {SAFE} and some $V \in \{view_i - 2, view_i - 1, view_i\}$:
invoke send_echo_three(V, v) ▷ a quorum of ECHO-2 message for v is received, issue ECHO-3 for v
upon cert(ECHO-3, $2f+1, V, v) = true$ for some $v \in$ Value ∪ {SAFE} and some view $V \geq view_i$ **and** $p_i \neq$ leader$(V+1)$:
$guard_i[V+1] \leftarrow v$
enter view $V+1$ ▷ active entry
upon cert(ECHO-3, $f+1, V, v) = true$ for some $v \in$ Value ∪ {SAFE} and some view $V \geq view_i$ **and** $p_i =$ leader$(V+1)$:
$guard_i[V+1] \leftarrow v$
if $v =$ SAFE **then** $value_to_propose_i[V+1] \leftarrow proposal_i$ ▷ if all values safe, propose own value
else $value_to_propose_i[V+1] \leftarrow v$ ▷ otherwise, must propose the guarded value
enter view $V+1$ ▷ active entry
upon $\langle$PREPARE, $V, \cdot\rangle$ is received **from** leader(V) **and** $V > view_i$ **and** cert(ECHO-2, $f+1, V-1, v) = true$ for some $v \in$ Value ∪ {SAFE}:
$guard_i[V] \leftarrow v$
enter view V ▷ active entry
upon cert(ECHO-1, $f+1, V, v) = true$ for some $v \in$ Value ∪ {SAFE} and some view $V \geq view_i$:
if $V > view_i$:
$guard_i[V] \leftarrow v$
passively enter view V
else:
invoke ensure_view_change$(view_i, v)$ ▷ skip to the view-change part of the current view
upon passively entering view V:
$view_i \leftarrow V$
invoke ensure_view_change$(V-2, guard_i[V])$
invoke ensure_view_change$(V-1, guard_i[V])$
invoke ensure_view_change$(V, guard_i[V])$

messages for v, each process *locks* on v (line 41) and broadcasts the lock via a VOTE-2 message. Finally, upon collecting $2f+1$ VOTE-2 messages for the same value v, each process decides v (line 44).

4.1.2 View Change. The view-change component is the most intricate part of the algorithm. Its goal is to determine, once view V completes, a value that is both *safe* (consistent with any decision that may have occurred in view V or earlier) and *live* (one that all correct processes will support) to carry into view $V+1$.

The challenge is delicate. If a value v was decided in view V, every correct process must learn v and adopt it as its guard for view $V+1$; supporting anything else would break agreement. If no decision occurred, processes must still converge on *some* value (or on the "all-safe" token SAFE), so that the next correct leader can make a proposal that will be universally supported.

VOTE-2 messages as the entry point to view change. The VOTE-2 message plays a dual role in Forget-IT: it serves both for deciding within the view core and as the entry point to the view-change

Algorithm 3 Forget-IT: Functions (for process p_i)

```
99:  - ensure_view_change(View V, Value ∪ {SAFE} v):  ▷ ensures view-change for view V begins
100:     vc_i[V] ← true
101:     if vote2_i[V] = ⊥: vote2_i[V] ← ⟨VOTE-2, V, ⊥, v⟩; broadcast vote2_i[V]
102: - send_echo_one(View V, Value ∪ {SAFE} v):  ▷ broadcasts ECHO-1 messages (at most two)
103:     if val_echo1_i[V][1] = ⊥:
104:         val_echo1_i[V][1] ← v; echo1_i[V][1] ← ⟨ECHO-1, V, v⟩; broadcast echo1_i[V][1]
105:     else if val_echo1_i[V][2] = ⊥ and val_echo1_i[V][1] ≠ SAFE and (v = SAFE or v ≠ val_echo1_i[V][1]):
106:         val_echo1_i[V][2] ← SAFE; echo1_i[V][2] ← ⟨ECHO-1, V, SAFE⟩; broadcast echo1_i[V][2]
107: - send_echo_two(View V, Value ∪ {SAFE} v):  ▷ broadcasts ECHO-2 messages (at most two)
108:     if val_echo2_i[V][1] = ⊥:
109:         val_echo2_i[V][1] ← v; echo2_i[V][1] ← ⟨ECHO-2, V, v⟩; broadcast echo2_i[V][1]
110:     else if val_echo2_i[V][2] = ⊥ and val_echo2_i[V][1] ≠ SAFE and (v = SAFE or v ≠ val_echo2_i[V][1]):
111:         val_echo2_i[V][2] ← SAFE; echo2_i[V][2] ← ⟨ECHO-2, V, SAFE⟩; broadcast echo2_i[V][2]
112: - send_echo_three(View V, Value ∪ {SAFE} v):  ▷ broadcasts ECHO-3 messages (at most two)
113:     if val_echo3_i[V][1] = ⊥:
114:         val_echo3_i[V][1] ← v; echo3_i[V][1] ← ⟨ECHO-3, V, v⟩; broadcast echo3_i[V][1]
115:     else if val_echo3_i[V][2] = ⊥ and val_echo3_i[V][1] ≠ SAFE and (v = SAFE or v ≠ val_echo3_i[V][1]):
116:         val_echo3_i[V][2] ← SAFE; echo3_i[V][2] ← ⟨ECHO-3, V, SAFE⟩; broadcast echo3_i[V][2]
117: - cert({ECHO-1, ECHO-2, ECHO-3} type, Integer threshold, View V, Value ∪ {SAFE} v):  ▷ checks whether a quorum for v is collected
118:     if v = SAFE:
119:         return there exists a set of threshold received ⟨type, V, SAFE⟩ messages
120:     else:
121:         ▷ for specific values, allow SAFE to count toward the threshold
122:         return there exists a set of threshold received ⟨type, V, ·⟩ messages, each carrying either v or SAFE
123: - divergent_guards(View V):  ▷ checks whether correct processes have different guards
124:     Let num_guards be the number of accepted VOTE-2 messages for view V
125:     Let G ∈ Value ∪ {SAFE} be the most frequent guard among the accepted VOTE-2 messages for view V
126:     ▷ detect if at least f + 1 honest processes have different guards
127:     return num_guards − #{⟨VOTE-2, V, ·, G⟩ is accepted} ≥ f + 1
```

procedure. To this end, each VOTE-2 message carries both the lock for the current view (which plays a role in both the view core and view change) and the guard for the current view (which plays a role only in view change). We deliberately reuse VOTE-2 rather than introducing a separate message type, as a dedicated view-change message would need to follow VOTE-2 (to capture any decision that may have occurred), adding an extra round of communication. To ensure that the view-change procedure always has enough VOTE-2 messages to work with, a timer fires 3Δ after entering the view (line 45): if a process has not yet broadcast a VOTE-2 through the standard "voting path" (because the leader is faulty or the network has not stabilized), it does so at this point, carrying its current lock (possibly $\perp$) and guard (line 48).

Before the view-change procedure can analyze VOTE-2 messages, they must be *accepted*. A VOTE-2 message with a $\perp$ lock is accepted immediately (line 24), since it makes no claim about a potential decision. A VOTE-2 message with a non-$\perp$ lock value v requires more care: it is accepted only after the process independently receives $f + 1$ $\langle$VOTE-1$, V, v\rangle$ messages (line 23). This acceptance condition is an information-theoretic substitute for digital signatures: if the sender is correct, the message will eventually be accepted by every correct process, since locking requires $2f + 1$ VOTE-1 messages, at least $f + 1$ from correct processes; if the sender is faulty, acceptance guarantees that at least one correct process voted for v, which (by the guarding mechanism) means that v is consistent with any decision in prior views.

To construct the view-change procedure, we face a key question: should processes trust the locks conveying information about view V, or the guards conveying information from prior views (and not view V)? The answer depends on whether view V reached a decision: if so, locks must take priority; if not, guards become relevant as they carry information about decisions from prior views (if any).

Determining whether a decision (might have) occurred in view V. Once a process accepts $2f + 1$ VOTE-2 messages for view V, it examines their lock values. A key invariant of our protocol is that two correct processes never lock on different values in the same view, which follows from standard quorum intersection. Therefore, if the accepted set contains two different non-$\perp$ lock values, at least one must originate from a faulty process, and the process can simply wait for more VOTE-2 messages before drawing conclusions.

Once the process has accepted $2f + 1$ VOTE-2 messages whose non-$\perp$ lock values all agree on some value ℓ (line 58), it records $decision_alert_i[V] \leftarrow true$ and $potential_decision_i[V] \leftarrow \ell$, signaling that ℓ may have been decided in view V (line 59). If every lock in the set is $\perp$, it records $decision_alert_i[V] \leftarrow false$, concluding that no decision occurred (line 60).

If the assessment concludes that a decision occurred, this conclusion is only tentative and can be revised as more VOTE-2 messages arrive. For instance, if the process later accepts a set of $2f + 1$ VOTE-2 messages all carrying $\perp$ locks (line 61), it updates $decision_alert_i[V] \leftarrow false$: no decision could have occurred if an entire quorum never locked. Similarly, if it accepts $2f + 1$ messages whose non-$\perp$ locks all differ from $potential_decision_i[V]$ (line 63), the process again concludes that no decision occurred: any decided value must appear in every set of $2f + 1$ VOTE-2 messages by quorum intersection, yet here two such sets report different unique non-$\perp$ locks, which implies no decision was reached.

Sending ECHO-1 messages. With the assessment of whether a decision occurred in hand, each process determines what value to echo for view V. There are three triggers for broadcasting an ECHO-1 message, the first level of echoing in our view-change procedure, which announces values deemed safe. Each trigger corresponds to a different source of safety information:

- *From locks* (line 68): Upon accepting a VOTE-2 message for view V with a non-$\perp$ lock value v, the process broadcasts an ECHO-1 message for v, provided that either $decision_alert_i[V] = false$ (no decision occurred, so v is safe regardless) or $potential_decision_i[V] = v$ (the tentative "decision estimate" matches v).
- *From guards* (line 70): If $decision_alert_i[V] = false$ and $f + 1$ accepted VOTE-2 messages for view V share the same guard value G, the process echoes G. Since at least one of the $f + 1$ senders is correct and no decision was reached in view V, G is a safe value inherited from a prior view.
- *From divergent guards* (line 72): If $decision_alert_i[V] = false$ and the divergent_guards function (line 123) detects that not all correct processes hold the same guard, the process concludes that all values are safe and broadcasts an ECHO-1 message for SAFE. The reasoning is that if only one value were truly safe, all correct processes would have adopted it as their guard prior to entering

V; diversity among honest guards therefore implies that every value is safe.

All ECHO-1 messages are routed through the send_echo_one function (line 102), which ensures that each process broadcasts at most two ECHO-1 messages per view: one for a specific value, and, if a second invocation arrives with a different value, a second one carrying SAFE (line 106). This design bounds the number of ECHO-1 messages to at most two per process per view.

The echo cascade. The ECHO-1 messages are amplified through two further rounds. Upon collecting $2f + 1$ ECHO-1 messages for view V, each carrying either v or SAFE, a process broadcasts an ECHO-2 message for v (line 74). Recall that the SAFE token acts as a wildcard: it counts toward the quorum for any given value, since it asserts that *every* value is safe. Upon collecting $2f + 1$ ECHO-2 messages under the same rule, the process broadcasts an ECHO-3 message (line 76). The send_echo_two and send_echo_three functions mirror the two-message-per-view design of send_echo_one.

One may wonder why three rounds of echoing are necessary, given that the safe value is already determined after ECHO-1. We use three rounds (instead of one) for two reasons: (1) maintaining constant persistent memory, and (2) minimizing latency. We elaborate on both in the informal analysis of Forget-IT (§4.2).

4.1.3 View Transitioning. The view-transitioning component governs when and how processes leave view V and enter view $V + 1$. We differentiate two types of entry into view $V + 1$ for a correct process: active (participating in the view core logic for deciding) and passive (skipping the view core and jumping immediately to the view-change procedure). We now describe each:

- *Active entry via ECHO-3 messages.* A non-leader process enters view $V + 1$ upon collecting $2f + 1$ ECHO-3 messages for view V, each carrying some value v or SAFE (line 77), while the leader of view $V + 1$ enters upon collecting only $f + 1$ such messages (line 80). This asymmetry is deliberate and key to achieving good latency: once any non-leader correct process enters view $V + 1$, a correct leader will follow within δ time, enabling a decision within 4δ of the first correct process's entry.
- *Active entry via PREPARE and ECHO-2 messages.* A process can also enter view $V+1$ upon receiving a $\langle \text{PREPARE}, V+1, \cdot \rangle$ message from the leader, provided it has independently collected $f + 1$ ECHO-2 messages for view V (line 85). The ECHO-2 certificate provides the process with a safe guard value for view $V+1$, allowing it to enter the new view without waiting for the full ECHO-3 cascade. This entrance is crucial for achieving robust good-case latency of 3δ: once the leader enters, all correct processes follow immediately.
- *Passive entry via ECHO-1 messages.* Finally, a process may *passively enter* view $V + 1$ upon receiving $f + 1$ ECHO-1 messages for view $V + 1$ (line 88). This path is designed for processes that have fallen behind and need to catch up; we explain the importance of this "quick catch-up" mechanism in §4.2. When a correct process receives $f + 1$ ECHO-1 message for view $V + 1$, it knows that the view-change procedure of view $V + 1$ has already started; consequently, it skips the view core and jumps directly to view change.

Setting the guard and the leader's proposal. Regardless of which path a process p_i takes into view $V + 1$, it determines its guard for view $V + 1$ ($guard_i[V + 1]$) based on the ECHO messages that triggered the transition: from ECHO-3 for ECHO-3-based active entry (lines 78 and 81), from ECHO-2 for PREPARE-based entry (line 86), or from ECHO-1 for passive entry (line 90). Similarly, if the leader of view $V + 1$ finds its guard to be SAFE (meaning no prior decision constrains its choice), it is free to propose its own input value (line 82). Otherwise, it must propose the specific value carried by the ECHO-3 messages (line 83), which equals its guard, ensuring consistency with any prior decision.

Epilogue. This closes the loop between all three components: the view-change procedure distills the outcome of view V into a safe and live value, the transitioning component propagates that value into the guards and the leader's proposal for view $V + 1$, and the view core ensures it is decided (assuming the leader is correct and GST has passed).

4.2 Proof Sketch

This section presents a proof sketch of Forget-IT's correctness and complexity. Recall that a formal proof can be found in the full version [3].

4.2.1 Correctness. We first give a proof sketch of the following theorem.

THEOREM 2 (FORGET-IT'S CORRECTNESS). *Given $n = 3f+1$, Forget-IT (Algorithm 1) is a correct implementation of the Byzantine agreement problem in the presence of a full-information, computationally unbounded adversary that can corrupt up to $f > 0$ processes.*

We now analyze each property separately.

Agreement. Suppose some correct process decides value v_{dec} in view V_{dec}, and this is the first decision in the execution (i.e., no correct process decides in any view smaller than V_{dec}). First, note that no other value can be decided in view V_{dec} due to standard quorum intersection.

We now establish that every correct process p_i enters view $V_{dec} + 1$ with its guard being v_{dec} ($guard_i[V_{dec} + 1] = v_{dec}$). When a decision occurs in view V_{dec}, at least $2f + 1$ correct processes broadcast $\langle \text{VOTE-2}, V_{dec}, v_{dec} \rangle$ messages (line 43). By quorum intersection, any set of $2f + 1$ VOTE-2 messages for view V_{dec} must contain at least one message with lock value v_{dec}. The key implication is that once a correct process p_i collects $2f + 1$ VOTE-2 messages and attempts to determine whether a decision occurred (line 58), it must conclude that one might have happened and that the value might be v_{dec} (by setting $decision_alert_i[V_{dec}] \leftarrow true$ and $potential_decision_i[V_{dec}] \leftarrow v_{dec}$ at line 59). Moreover, it becomes impossible for p_i to receive $2f + 1$ VOTE-2 messages with all $\perp$ locks (blocking the rule at line 61) or $2f + 1$ messages with a unique non-$\perp$ lock $\ell \neq v_{dec}$ (blocking the rule at line 63), since v_{dec} must appear in any quorum of $2f + 1$ VOTE-2 messages for view V_{dec}.

Therefore, every correct process executing the view-change procedure for view V_{dec} concludes that a decision might have occurred and identifies v_{dec} as the decided value. This ensures that process p_i can only send an ECHO-1 message carrying v_{dec}: with $potential_decision_i[V_{dec}] = v_{dec}$ and $decision_alert_i[V_{dec}] = true$, the rule at line 67 triggers exclusively for v_{dec}, while the other two ECHO-1 triggers (lines 69 and 71) cannot activate. This cascades

through the echo layers: all ECHO-2 and ECHO-3 messages broadcast during view V_{dec}'s view-change procedure carry v_{dec} (lines 74 and 76). Consequently, every correct process p_i, regardless of which transition path it takes into view $V_{dec}+1$, sets its guard for view $V_{dec}+1$ to v_{dec} ($guard_i[V_{dec}+1] \leftarrow v_{dec}$), which ensures that no value other than v_{dec} can be decided in view $V_{dec}+1$.

Finally, once all correct processes have $guard_i[V_{dec}+1] = v_{dec}$, no correct process can broadcast an ECHO-1 message for any value other than v_{dec} during view $V_{dec}+1$'s view-change procedure. To see why, recall that there are three triggers for broadcasting ECHO-1: first, upon accepting a VOTE-2 message with a non-$\perp$ lock (line 67), the lock value must be v_{dec} since any lock is backed by at least $f+1$ VOTE-1 messages for view $V_{dec}+1$, and all correct processes only support v_{dec} in that view; second, upon accepting $f+1$ VOTE-2 messages carrying the same guard (line 69), this guard must be v_{dec} since all correct processes p_i have $guard_i[V_{dec}+1] = v_{dec}$; third, upon detecting that not all correct processes hold the same guard (line 71), which cannot occur because all correct processes do have the same guard (namely, v_{dec}). As a result, all ECHO-1 messages in view $V_{dec}+1$ carry v_{dec}, which propagates through ECHO-2 and ECHO-3, ensuring every correct process enters view $V_{dec}+2$ with v_{dec} as its guard. By induction, we have $guard_i[V'] = v_{dec}$ for all views $V' > V_{dec}$ and all correct processes p_i, which prevents any decision on a value other than v_{dec}.

Termination. For termination, consider the first view V after GST with a correct leader p_ℓ. To enter view V, leader p_ℓ must receive $f+1$ ECHO-3 messages from view $V-1$, each carrying either some value v or the special "all-safe" value SAFE (line 80). At least one correct process must have sent such an ECHO-3 message, which requires that process to have collected $2f+1$ ECHO-2 messages for either v or SAFE (line 75). Within δ time, all correct processes receive $f+1$ ECHO-2 messages for either v or SAFE. Upon receiving the leader's PREPARE message with this certificate in hand (line 85), each correct process enters view V and supports the proposal: the certificate demonstrates that the leader's value is safe, allowing the process to proceed with voting. All correct processes then broadcast VOTE-1 messages (line 38), followed by VOTE-2 messages (line 42), leading to a decision in view V within 3δ of the leader's entry (line 44).

Non-triviality. If GST = 0, all processes are correct, and they all propose the same value, then the first view must succeed and this value must be decided.

4.2.2 Complexity. Next, we provide a proof sketch of the following theorem.

THEOREM 3 (FORGET-IT'S COMPLEXITY). *Given $n = 3f+1$, the following holds for Forget-IT (Algorithm 1) in the presence of a full-information, computationally unbounded adversary that can corrupt up to $f > 0$ processes:*

- *Robust good-case latency:* 3δ
- *View length:* 7Δ
- *Message complexity per view:* $O(n^2)$
- *Memory complexity:* $O(1)$ *words (per correct process)*

The robust good-case latency of 3δ follows immediately from the analysis of termination (and implies non-robust good-case latency of 3δ), so here we focus on the other performance metrics.

Message complexity per view. In each view, every correct process broadcasts at most a constant number of messages. The VOTE-1 and VOTE-2 messages are each broadcast at most once per view, as each correct process p_i stores them in $vote1_i[\cdot]$ and $vote2_i[\cdot]$ and conditions broadcasts on these being $\perp$. The functions send_echo_one, send_echo_two, and send_echo_three each broadcast at most twice per view—once for a specific value and once upon upgrading to SAFE—yielding at most two ECHO-1, two ECHO-2, and two ECHO-3 messages. With the single PREPARE message from the leader, each correct process broadcasts at most 9 messages per view. Since broadcasts reach all n processes and there are at most n correct processes, the total message complexity is $O(n^2)$ per view.

View length & memory complexity. Once all correct processes are in the same post-GST view V, they complete that view and transition to view $V+1$ within 7Δ time. This bound follows from the timeout mechanism: all correct processes broadcast their VOTE-2 messages by time 3Δ (line 48), which are received by time 4Δ, triggering ECHO-1 messages. The subsequent ECHO-2 and ECHO-3 cascades complete within an additional 3Δ, yielding the 7Δ bound. However, what guarantees that all correct processes will actually participate in view V? This concern is critical because our protocol maintains constant memory by tracking only the last three views (current, previous, and the one before that), meaning a process stops participating in views outside this window. We must verify that all correct processes overlap for sufficiently long in view V to complete it, despite this memory restriction.

Suppose the first correct process enters view V at time τ. A key property of Forget-IT —enabled by the multiple levels of echoing—ensures that all correct processes enter view V (or a later view) by time $\tau+5\Delta$. Concretely, the multiple echo levels guarantee that within Δ time, every correct process is brought to at least view $V-1$ (via the passive entry mechanism at line 88), from which point it takes at most 4Δ to complete the view-change procedure of view $V-1$ and transition to view V. This rapid convergence property is essential for constant memory: without multiple echo levels, processes could diverge by $O(f)$ views due to a sequence of faulty leaders. In such a scenario, each correct process would need to track the last $O(f)$ views to ensure progress for stragglers, requiring $O(f)$ memory instead of $O(1)$.

On the other end, the 4Δ timeout (line 65) bounds how quickly processes can advance: by ensuring that no correct process broadcasts any ECHO-1 message for view V before time $\tau+4\Delta$, it guarantees that no process enters view $V+1$ before time $\tau+4\Delta$. More generally, since each view requires at least 4Δ to complete (due to the timeout), no correct process can enter any view beyond $V+2$ before time $\tau+12\Delta$. This bounded advancement rate, combined with the three-view tracking window, ensures that no correct process "forgets" view V before all others have had sufficient time to participate in it. Indeed, the last correct process enters view V by time $\tau+5\Delta$, while no correct process abandons view V before time $\tau+12\Delta$. Therefore, all correct processes overlap in their participation in view V (specifically, in its view-change phase) during the interval $[\tau+5\Delta, \tau+12\Delta]$, providing a 7Δ window sufficient for completion.

References

[1] Ittai Abraham, Yonatan Amit, and Danny Dolev. 2004. Optimal Resilience Asynchronous Approximate Agreement. In *Principles of Distributed Systems, 8th International Conference, OPODIS 2004, Grenoble, France, December 15-17, 2004, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 3544)*, Teruo Higashino (Ed.). Springer, 229–239. doi:10.1007/11516798_17

[2] Ittai Abraham, Naama Ben-David, and Sravya Yandamuri. 2022. Efficient and Adaptively Secure Asynchronous Binary Agreement via Binding Crusader Agreement. In *PODC '22: ACM Symposium on Principles of Distributed Computing, Salerno, Italy, July 25 - 29, 2022*, Alessia Milani and Philipp Woelfel (Eds.). ACM, 381–391. doi:10.1145/3519270.3538426

[3] Ittai Abraham, Sourav Das, Yuval Efron, and Jovan Komatovic. 2026. Forget-IT: Optimal Good-Case Latency For Information-Theoretic BFT. *Cryptology ePrint Archive* (2026).

[4] Ittai Abraham, Srinivas Devadas, Kartik Nayak, and Ling Ren. 2017. Brief Announcement: Practical Synchronous Byzantine Consensus. In *31st International Symposium on Distributed Computing (DISC 2017)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.

[5] Ittai Abraham, Yuval Efron, and Ling Ren. 2025. The Latency Cost Of Censorship Resistance. *IACR Cryptol. ePrint Arch.* (2025), 2136.

[6] Ittai Abraham, Kartik Nayak, Ling Ren, and Zhuolun Xiang. 2021. Good-case Latency of Byzantine Broadcast: a Complete Categorization. In *PODC '21: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, July 26-30, 2021*, Avery Miller, Keren Censor-Hillel, and Janne H. Korhonen (Eds.). ACM, 331–341. doi:10.1145/3465084.3467899

[7] Ittai Abraham, Kartik Nayak, and Nibesh Shrestha. 2021. Optimal Good-Case Latency for Rotating Leader Synchronous BFT. In *25th International Conference on Principles of Distributed Systems, OPODIS 2021, Strasbourg, France, December 13-15, 2021 (LIPIcs, Vol. 217)*, Quentin Bramas, Vincent Gramoli, and Alessia Milani (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 27:1–27:19. doi:10.4230/LIPICS.OPODIS.2021.27

[8] Ittai Abraham, Ling Ren, and Zhuolun Xiang. 2021. Good-Case and Bad-Case Latency of Unauthenticated Byzantine Broadcast: A Complete Categorization. In *25th International Conference on Principles of Distributed Systems, OPODIS 2021, Strasbourg, France, December 13-15, 2021 (LIPIcs, Vol. 217)*, Quentin Bramas, Vincent Gramoli, and Alessia Milani (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 5:1–5:20. doi:10.4230/LIPICS.OPODIS.2021.5

[9] Ittai Abraham and Gilad Stern. 2021. Information Theoretic HotStuff. In *24th International Conference on Principles of Distributed Systems (OPODIS 2020)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 11–1.

[10] Ittai Abraham and Gilad Stern. 2021. Information Theoretic HotStuff (IT-HS) Part One. https://decentralizedthoughts.github.io/2021-09-20-information-theoretic-hotstuff-it-hs-part-one/.

[11] Timothé Albouy, Davide Frey, Michel Raynal, and François Taïani. 2024. Good-case early-stopping latency of synchronous byzantine reliable broadcast: the deterministic case. *Distributed Computing* 37, 2 (2024), 121–143.

[12] Hagit Attiya and Jennifer L. Welch. 2023. Brief Announcement: Multi-Valued Connected Consensus: A New Perspective on Crusader Agreement and Adopt-Commit. In *37th International Symposium on Distributed Computing, DISC 2023, October 10-12, 2023, L'Aquila, Italy (LIPIcs, Vol. 281)*, Rotem Oshman (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 36:1–36:7. doi:10.4230/LIPICS.DISC.2023.36

[13] Michael Ben-Or. 1983. Another Advantage of Free Choice: Completely Asynchronous Agreement Protocols (Extended Abstract). In *PODC*. ACM, 27–30.

[14] Gabriel Bracha. 1987. Asynchronous Byzantine agreement protocols. *Information and computation* 75, 2 (1987), 130–143.

[15] Ethan Buchman. 2016. *Tendermint: Byzantine Fault Tolerance in the Age of Blockchains*. Ph. D. Dissertation. University of Guelph.

[16] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. 2001. Secure and Efficient Asynchronous Broadcast Protocols. In *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings (Lecture Notes in Computer Science, Vol. 2139)*, Joe Kilian (Ed.). Springer, 524–541. doi:10.1007/3-540-44647-8_31

[17] Benjamin Y. Chan and Rafael Pass. 2023. Simplex Consensus: A Simple and Fast Consensus Protocol. In *TCC (4) (Lecture Notes in Computer Science, Vol. 14372)*. Springer, 452–479.

[18] Jinyuan Chen. 2024. OciorMVBA: Near-Optimal Error-Free Asynchronous MVBA. arXiv:2501.00214v1 [cs.CR]. arXiv:2501.00214v1 [cs.CR] http://arxiv.org/abs/2501.00214v1

[19] Pierre Civit, Muhammad Ayaz Dzulfikar, Seth Gilbert, Vincent Gramoli, Rachid Guerraoui, Jovan Komatovic, and Manuel Vidigueira. 2022. Byzantine Consensus is $\Theta(n^2)$: The Dolev-Reischuk Bound is Tight even in Partial Synchrony!. In *36th International Symposium on Distributed Computing (DISC 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 246)*, Christian Scheideler (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 14:1–14:21. doi:10.4230/LIPIcs.DISC.2022.14

[20] Pierre Civit, Seth Gilbert, and Vincent Gramoli. 2021. Polygraph: Accountable Byzantine Agreement. In *Proceedings of the 41st IEEE International Conference on Distributed Computing Systems (ICDCS'21)*.

[21] Pierre Civit, Seth Gilbert, Rachid Guerraoui, Jovan Komatovic, and Manuel Vidigueira. 2023. On the Validity of Consensus. In *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing*. 332–343.

[22] Tyler Crain, Vincent Gramoli, Mikel Larrea, and Michel Raynal. 2018. DBFT: Efficient Leaderless Byzantine Consensus and its Applications to Blockchains. In *Proceedings of the 17th IEEE International Symposium on Network Computing and Applications (NCA'18)*. IEEE.

[23] Miguel Oom Temudo de Castro. 2000. *Practical Byzantine fault tolerance*. Ph. D. Dissertation. Massachusetts Institute of Technology, Cambridge, MA, USA. https://hdl.handle.net/1721.1/86581

[24] Danny Dolev, Rüdiger Reischuk, and H. Raymond Strong. 1990. Early Stopping in Byzantine Agreement. *J. ACM* 37, 4 (1990), 720–741. doi:10.1145/96559.96565

[25] Danny Dolev and H. Raymond Strong. 1983. Authenticated Algorithms for Byzantine Agreement. *SIAM J. Comput.* 12, 4 (1983), 656–666.

[26] Manu Drijvers, Tim Gretler, Yotam Harchol, Tobias Klenze, Ognjen Maric, Stefan Neamtu, Yvonne-Anne Pignolet, Rostislav Rumenov, Daniel Sharifi, and Victor Shoup. 2024. A New Broadcast Primitive for BFT Protocols. *CoRR* abs/2410.22080 (2024). arXiv:2410.22080 doi:10.48550/ARXIV.2410.22080

[27] Sisi Duan, Xin Wang, and Haibin Zhang. 2023. FIN: Practical Signature-Free Asynchronous Common Subset in Constant Time. In *CCS*. ACM, 815–829.

[28] Yuval Efron, Joachim Neu, Ling Ren, and Ertem Nusret Tas. 2025. Optimal Good-Case Latency for Sleepy Consensus. *CoRR* abs/2510.06023 (2025). arXiv:2510.06023 doi:10.48550/ARXIV.2510.06023

[29] Pesech Feldman and Silvio Micali. 1997. An Optimal Probabilistic Protocol for Synchronous Byzantine Agreement. *SIAM J. Comput.* 26, 4 (1997), 873–933. doi:10.1137/S0097539790187084

[30] Michael J Fischer, Nancy A Lynch, and Michael S Paterson. 1985. Impossibility of Distributed Consensus with One Faulty Process. *Journal of the ACM (JACM)* 32, 2 (1985), 374–382.

[31] Matthias Fitzi and Juan A Garay. 2003. Efficient Player-Optimal Protocols for Strong and Differential Consensus. In *Proceedings of the twenty-second annual symposium on Principles of distributed computing*. 211–220.

[32] Kim P. Kihlstrom, Louise E. Moser, and Peter M. Melliar-Smith. 2003. Byzantine Fault Detectors for Solving Consensus. *British Computer Society* (2003).

[33] Jovan Komatovic, Joachim Neu, and Tim Roughgarden. 2024. Toward Optimal-Complexity Hash-Based Asynchronous MVBA with Optimal Resilience. *CoRR* abs/2410.12755 (2024). arXiv:2410.12755 doi:10.48550/ARXIV.2410.12755

[34] Leslie Lamport. 1983. The weak Byzantine generals problem. *Journal of the ACM (JACM)* 30, 3 (1983), 668–676.

[35] Leslie Lamport, Robert Shostak, and Marshall Pease. 1982. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems* 4, 3 (1982), 382–401.

[36] Yuan Lu, Zhenliang Lu, Qiang Tang, and Guiling Wang. 2020. Dumbo-MVBA: Optimal Multi-Valued Validated Asynchronous Byzantine Agreement, Revisited. In *PODC*. ACM, 129–138.

[37] Jean-Philippe Martin and Lorenzo Alvisi. 2006. Fast Byzantine Consensus. *IEEE Trans. Dependable Secur. Comput.* 3, 3 (July 2006), 202–215. doi:10.1109/TDSC.2006.35

[38] Darya Melnyk and Roger Wattenhofer. 2018. Byzantine Agreement with Interval Validity. In *2018 IEEE 37th Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 251–260.

[39] Achour Mostéfaoui and Michel Raynal. 2017. Signature-Free Asynchronous Byzantine Systems: From Multivalued to Binary Consensus with t< n/3, $O(n^2)$ Messages, and Constant Time. *Acta Informatica* 54, 5 (2017), 501–520. doi:10.1007/S00236-016-0269-Y

[40] Marshall C. Pease, Robert E. Shostak, and Leslie Lamport. 1980. Reaching Agreement in the Presence of Faults. *J. ACM* 27, 2 (1980), 228–234. doi:10.1145/322186.322188

[41] Michael O. Rabin. 1983. Randomized Byzantine Generals. In *FOCS*. IEEE Computer Society, 403–409.

[42] Hin-Sing Siu, Yeh-Hao Chin, and Wei-Pang Yang. 1998. Reaching strong consensus in the presence of mixed failure types. *Information Sciences* 108, 1-4 (1998), 157–180.

[43] David Stolz and Roger Wattenhofer. 2016. Byzantine Agreement with Median Validity. In *19th International Conference on Principles of Distributed Systems (OPODIS 2015)*, Vol. 46. Schloss Dagstuhl–Leibniz-Zentrum für Informatik GmbH, 22.

[44] Maofan Yin, Dahlia Malkhi, Michael K Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT Consensus with Linearity and Responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*. 347–356.

[45] Qianyu Yu, Giuliano Losa, and Xuechao Wang. 2024. TetraBFT: Reducing Latency of Unauthenticated, Responsive BFT Consensus. In *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing, PODC 2024, Nantes, France, June 17-21, 2024*, Ran Gelles, Dennis Olivetti, and Petr Kuznetsov (Eds.). ACM, 257–267. doi:10.1145/3662158.3662783